

Competitive Programming

Section 19 Editorial: Dijkstra And Weighted Modeling

Competitive Programming Course

2026-07-26

Table of contents

1	Editorial overview	4
2	Dijkstra reminder	5
3	A. Weighted Route Queries	6
3.1	Restatement	6
3.2	Observations	6
3.3	Algorithm	6
3.4	Correctness proof	6
3.5	Complexity	6
3.6	Full C++ solution	7
3.7	Full Python solution	8
4	B. Grid Toll Path	9
4.1	Restatement	9
4.2	Observations	9
4.3	Algorithm	9
4.4	Correctness	10
4.5	Complexity	10
4.6	Full C++ solution	10
4.7	Full Python solution	11
5	C. Discount Route	12
5.1	Restatement	12
5.2	Observations	12
5.3	Algorithm	12
5.4	Correctness	13
5.5	Complexity	13
5.6	Full C++ solution	13
5.7	Full Python solution	14
6	D. Even-Hop Route	16
6.1	Restatement	16
6.2	Constraints and traps	16
6.3	Observation and walk-through	16
6.4	Algorithm	16
6.5	Correctness proof	17
6.6	Complexity	17
6.7	C++ implementation notes	17
6.8	Python implementation notes	18
6.9	Full C++ solution	18
6.10	Full Python solution	19
6.11	Common wrong answers	19
6.12	Tests that matter	20

7	Common wrong answers	21
8	Stress testing	22
9	What to carry forward	23

1 Editorial overview

Problem	Model
A. Weighted Route Queries	directed graph Dijkstra
B. Grid Toll Path	grid cells as weighted graph
C. Discount Route	two-layer state graph
D. Even-Hop Route	vertex and path-length parity state

2 Dijkstra reminder

Dijkstra maintains tentative distances and always expands the smallest tentative distance first. With nonnegative weights, the first non-stale pop of a state finalizes its shortest distance.

Stale heap entries happen because we push improved distances instead of decreasing keys:

```
if popped_distance != dist[state]:  
    skip it
```

3 A. Weighted Route Queries

3.1 Restatement

Given a directed weighted graph, source s , and queries, print the shortest distance from s to each queried vertex or -1 .

3.2 Observations

All queries share the same source, so run Dijkstra once and answer queries by array lookup.

3.3 Algorithm

```
build adjacency list
run Dijkstra from s
for each query x:
    print dist[x] or -1
```

3.4 Correctness proof

Lemma. When Dijkstra pops a non-stale pair for vertex u , $\text{dist}[u]$ is the shortest distance from s to u .

Proof. Suppose a shorter path to u existed. Let x be the first vertex on that path not finalized yet, and p its predecessor. Vertex p was finalized earlier, so edge $p \rightarrow x$ would have relaxed x to at most the shorter path prefix. Since all weights are nonnegative, that distance would be no larger than the alleged shorter distance to u , contradicting that u was the smallest popped state. $\square$

Theorem. Every printed query answer is correct.

Proof. By the lemma, Dijkstra computes every reachable shortest distance. Unreached vertices keep the sentinel value and are correctly printed as -1 . $\square$

3.5 Complexity

```
Time:  $O((n + m) \log n)$ 
Memory:  $O(n + m)$ 
```

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s, qn;
    if (!(cin >> n >> m >> s >> qn)) return 0;
    --s;
    vector<vector<pair<int, long long>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; long long w;
        cin >> u >> v >> w;
        adj[u - 1].push_back({v - 1, w});
    }
    vector<int> queries(qn);
    for (int &x : queries) { cin >> x; --x; }

    const long long INF = (1LL << 62);
    vector<long long> dist(n, INF);
    priority_queue<pair<long long, int>,
                  vector<pair<long long, int>>,
                  greater<pair<long long, int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            long long nd = d + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                pq.push({nd, v});
            }
        }
    }
    for (int i = 0; i < qn; i++) {
        if (i) cout << ' ';
        cout << (dist[queries[i]] == INF ? -1 : dist[queries[i]]);
    }
    cout << '\n';
    return 0;
}
```

3.7 Full Python solution

```
import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it); s = next(it) - 1; qn = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1; v = next(it) - 1; w = next(it)
        adj[u].append((v, w))
    queries = [next(it) - 1 for _ in range(qn)]

    inf = 10**30
    dist = [inf] * n
    dist[s] = 0
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d != dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    print(" ".join(str(-1 if dist[x] == inf else dist[x]) for x in queries))

if __name__ == "__main__":
    main()
```

4 B. Grid Toll Path

4.1 Restatement

Find the minimum total toll from the top-left cell to the bottom-right cell. Entering a cell costs that cell's toll, and the start cell is included.

4.2 Observations

Each cell is a graph vertex. Moving to a neighboring cell is a directed edge whose cost is the neighbor's toll. Costs are nonnegative, so Dijkstra applies.

The state and its meaning are deliberately simple:

State	Meaning
(r, c)	we are standing in cell (r, c)
$\text{dist}[r][c]$	least total toll of a route that has entered (r, c)

Initialize $\text{dist}[0][0]$ to $\text{cost}[0][0]$, not zero: the statement charges for the starting cell. From (r, c) , each in-bounds four-neighbor (nr, nc) has transition cost $\text{cost}[nr][nc]$. There is no separate cost for leaving a cell.

On the sample, Dijkstra first gives the starting cell distance 1. The move down gives $(1, 0)$ distance 2, whereas the move right gives $(0, 1)$ distance 10. Following the low-toll corridor reaches the goal at 6; the expensive top route remains in the heap but cannot improve that answer.

4.3 Algorithm

```
dist[*][*] = infinity
dist[0][0] = cost[0][0]
push (cost[0][0], 0, 0) into a min-heap

while the heap is nonempty:
    (d, r, c) = pop its smallest entry
    if d != dist[r][c]: continue          # stale entry
    for each of the four in-bounds neighbors (nr, nc):
        nd = d + cost[nr][nc]
        if nd < dist[nr][nc]:
            dist[nr][nc] = nd
            push (nd, nr, nc)
```

```
print dist[n-1][m-1]
```

4.4 Correctness

Dijkstra's invariant applies directly to cell states. Every legal grid walk is a path in the graph, and every graph path is a legal grid walk. Therefore the shortest graph distance to $(n-1, m-1)$ is exactly the minimum toll path.

4.5 Complexity

```
Time:  $O(n*m*\log(n*m))$   
Memory:  $O(n*m)$ 
```

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<long long>> cost(n, vector<long long>(m));
    for (auto &row : cost) for (long long &x : row) cin >> x;
    const long long INF = (1LL << 62);
    vector<vector<long long>> dist(n, vector<long long>(m, INF));
    using State = tuple<long long, int, int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    dist[0][0] = cost[0][0];
    pq.push({dist[0][0], 0, 0});
    int dr[4] = {1, -1, 0, 0};
    int dc[4] = {0, 0, 1, -1};
    while (!pq.empty()) {
        auto [d, r, c] = pq.top(); pq.pop();
        if (d != dist[r][c]) continue;
        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            long long nd = d + cost[nr][nc];
            if (nd < dist[nr][nc]) {
                dist[nr][nc] = nd;
                pq.push({nd, nr, nc});
            }
        }
    }
}
```

```

    }
}
cout << dist[n - 1][m - 1] << '\n';
return 0;
}

```

4.7 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it)
    cost = [[next(it) for _ in range(m)] for _ in range(n)]
    inf = 10**30
    dist = [[inf] * m for _ in range(n)]
    dist[0][0] = cost[0][0]
    heap = [(cost[0][0], 0, 0)]
    while heap:
        d, r, c = heapq.heappop(heap)
        if d != dist[r][c]:
            continue
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m:
                nd = d + cost[nr][nc]
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    heapq.heappush(heap, (nd, nr, nc))
    print(dist[n - 1][m - 1])

if __name__ == "__main__":
    main()

```

5 C. Discount Route

5.1 Restatement

Find the cheapest path from 1 to n when one edge may be discounted to $\text{floor}(w/2)$.

5.2 Observations

The vertex alone is not enough; we must know whether the discount was already used. Use states:

```
(u,0): at u, discount unused
(u,1): at u, discount used
```

All expanded edges have nonnegative weights, so Dijkstra works.

Make the transition table before coding. It prevents the common error of either losing the unused-discount route or taking the discount twice.

Current state	Traverse $u \rightarrow v$ of weight w	Next state	Added cost
(u,0)	do not use discount	(v,0)	w
(u,0)	use discount now	(v,1)	$\text{floor}(w/2)$
(u,1)	ordinary traversal	(v,1)	w

On the sample, the cheap-looking route $1 \rightarrow 2 \rightarrow 4$ costs $10 + 5 = 15$ if the discount is used on its second edge. The alternative $1 \rightarrow 3 \rightarrow 4$ costs $50 + 1 = 51$ after discounting $1 \rightarrow 3$. The discount belongs in the state, because comparing only the two arrivals at a vertex would discard this choice.

5.3 Algorithm

```
dist[u][used] = infinity for every vertex u and used in {0, 1}
dist[1][0] = 0
push (0, 1, 0) into a min-heap

while the heap is nonempty:
    (d, u, used) = pop its smallest entry
    if d != dist[u][used]: continue
    for each directed edge  $u \rightarrow v$  of weight  $w$ :
```

```

    relax (v, used) with d + w
    if used == 0:
        relax (v, 1) with d + floor(w / 2)

answer = min(dist[n][0], dist[n][1])
print -1 if answer is infinity, otherwise answer

```

5.4 Correctness

Every valid original route corresponds to one path in the expanded graph: normal edges stay in the same layer, and the discounted edge moves from layer 0 to layer 1. Conversely, every expanded-graph path describes a valid route using the discount at most once. Dijkstra therefore finds the optimal route.

5.5 Complexity

```

States: 2n; transitions: at most 3m
Time:   O((n + m) log n)
Memory: O(n + m)

```

5.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<pair<int, long long>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; long long w;
        cin >> u >> v >> w;
        adj[u - 1].push_back({v - 1, w});
    }
    const long long INF = (1LL << 62);
    vector<array<long long, 2>> dist(n, {INF, INF});
    using State = tuple<long long, int, int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    dist[0][0] = 0;
    pq.push({0, 0, 0});
    while (!pq.empty()) {
        auto [d, u, used] = pq.top(); pq.pop();

```

```

    if (d != dist[u][used]) continue;
    for (auto [v, w] : adj[u]) {
        long long nd = d + w;
        if (nd < dist[v][used]) {
            dist[v][used] = nd;
            pq.push({nd, v, used});
        }
        if (!used) {
            nd = d + w / 2;
            if (nd < dist[v][1]) {
                dist[v][1] = nd;
                pq.push({nd, v, 1});
            }
        }
    }
}

long long ans = min(dist[n - 1][0], dist[n - 1][1]);
cout << (ans == INF ? -1 : ans) << '\n';
return 0;
}

```

5.7 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1; v = next(it) - 1; w = next(it)
        adj[u].append((v, w))
    inf = 10**30
    dist = [[inf, inf] for _ in range(n)]
    dist[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        d, u, used = heapq.heappop(heap)
        if d != dist[u][used]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v][used]:

```

```

        dist[v][used] = nd
        heapq.heappush(heap, (nd, v, used))
    if used == 0:
        nd = d + w // 2
        if nd < dist[v][1]:
            dist[v][1] = nd
            heapq.heappush(heap, (nd, v, 1))
    ans = min(dist[n - 1])
    print(-1 if ans == inf else ans)

if __name__ == "__main__":
    main()

```

6 D. Even-Hop Route

6.1 Restatement

Find the least-cost directed route from vertex 1 to vertex n , subject to one extra condition: the route must use an even number of roads. Roads have nonnegative costs, vertices and roads may be revisited, and there can be no valid even-hop route even when an ordinary route exists.

6.2 Constraints and traps

n and m can both be 200000, so enumerating routes or storing a distance for every possible number of hops is impossible. The parity of the hop count, not the exact count, is all that matters. A one-road route to the destination is invalid, while the zero-road route from 1 to itself is valid and costs zero.

Do not keep just one best distance per vertex. A cheaper arrival with odd parity might be unusable, while a more expensive even-parity arrival is the one that can answer the problem.

6.3 Observation and walk-through

For every original vertex u , make two states:

```
(u, 0): u has been reached using an even number of roads  
(u, 1): u has been reached using an odd number of roads
```

Every road flips the parity. Thus a directed road $u \rightarrow v$ of cost w becomes two state transitions:

```
(u, 0) -> (v, 1), cost w  
(u, 1) -> (v, 0), cost w
```

Start in $(1, 0)$ and seek $(n, 0)$. This is an ordinary nonnegative shortest-path problem on $2n$ states.

In the sample, the tempting route $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ costs 12, but has three roads and therefore ends in state $(5, 1)$. The route $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ costs $3 + 1 + 1 + 1 = 6$ and uses four roads, so it reaches $(5, 0)$ and is valid.

6.4 Algorithm

```

dist[u][parity] = infinity for all vertices and both parities
dist[1][0] = 0
push (0, 1, 0) into a min-heap

while heap is not empty:
    pop (cost, u, parity)
    if cost is stale: continue
    for every road u -> v with weight w:
        next_parity = parity xor 1
        relax dist[v][next_parity] with cost + w

print dist[n][0], or -1 if it stayed infinite

```

6.5 Correctness proof

Lemma 1. Every route from 1 to u using parity p corresponds to a path from state $(1, 0)$ to state (u, p) of the same cost, and vice versa.

Proof. Starting at parity zero, each original road toggles parity exactly once. The expanded graph has precisely the matching parity-toggling transition with the same weight for every original road. Applying this correspondence road by road maps an original route to a state path of equal cost. Reversing the mapping reconstructs an original route from any state path. $\square$

Lemma 2. Dijkstra computes the minimum cost for every reachable parity state.

Proof. By construction every expanded transition has the original road's nonnegative cost. Dijkstra's usual smallest-non-stale-pop invariant therefore applies to the expanded graph, so each finalized state distance is shortest. $\square$

Theorem. The algorithm prints the minimum cost of a valid even-hop route, or -1 exactly when none exists.

Proof. By Lemma 1, valid routes are exactly paths from $(1, 0)$ to $(n, 0)$, with equal costs. Lemma 2 finds the cheapest such state path. If $(n, 0)$ is unreachable, Lemma 1 says no valid route exists, so -1 is correct. $\square$

6.6 Complexity

The expanded graph has $2n$ states and $2m$ transitions.

```

Time:     $O((n + m) \log n)$ 
Memory:  $O(n + m)$ 

```

6.7 C++ implementation notes

Use `array<long long, 2>` for the two state distances per vertex. The bitwise expression `parity ^ 1` toggles 0 to 1 and 1 to 0. Use `long long` for all path costs because a shortest route can exceed 32-bit range.

6.8 Python implementation notes

Store distances as `dist[vertex][parity]`, and push `(cost, vertex, parity)` tuples into `heapq`. Python integers do not overflow, but the same stale-entry check is still essential for both correctness and running time.

6.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, long long>>> graph(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        graph[u - 1].push_back({v - 1, w});
    }

    const long long INF = (1LL << 62);
    vector<array<long long, 2>> dist(n, {INF, INF});
    using State = tuple<long long, int, int>;
    priority_queue<State, vector<State>, greater<State>> heap;
    dist[0][0] = 0;
    heap.push({0, 0, 0});

    while (!heap.empty()) {
        auto [cost, u, parity] = heap.top();
        heap.pop();
        if (cost != dist[u][parity]) continue;
        for (auto [v, weight] : graph[u]) {
            int next_parity = parity ^ 1;
            long long next_cost = cost + weight;
            if (next_cost < dist[v][next_parity]) {
                dist[v][next_parity] = next_cost;
                heap.push({next_cost, v, next_parity});
            }
        }
    }

    cout << (dist[n - 1][0] == INF ? -1 : dist[n - 1][0]) << '\n';
}
```

6.10 Full Python solution

```
import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u, v, weight = data[index:index + 3]
        index += 3
        graph[u - 1].append((v - 1, weight))

    inf = 10**30
    dist = [[inf, inf] for _ in range(n)]
    dist[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        cost, u, parity = heapq.heappop(heap)
        if cost != dist[u][parity]:
            continue
        for v, weight in graph[u]:
            next_parity = parity ^ 1
            next_cost = cost + weight
            if next_cost < dist[v][next_parity]:
                dist[v][next_parity] = next_cost
                heapq.heappush(heap, (next_cost, v, next_parity))

    print(-1 if dist[n - 1][0] == inf else dist[n - 1][0])

if __name__ == "__main__":
    main()
```

6.11 Common wrong answers

- Running ordinary Dijkstra with one distance per vertex, which loses parity.
- Printing the cheaper of `dist[n][0]` and `dist[n][1]`; only even parity is valid.
- Starting at (1, 1) instead of (1, 0).
- Counting total edge weight parity instead of the number of roads.
- Marking a state final when it is pushed into the heap rather than skipping stale popped entries.

6.12 Tests that matter

- $n = 1$ with no roads, whose answer is zero;
- a one-road path to the target, which is unreachable with even parity;
- an odd-hop cheap route and an even-hop more expensive route;
- zero-cost cycles that change parity without changing the cost;
- parallel roads of different costs;
- a long graph with 200000 roads, to test state count and heap behavior.

7 Common wrong answers

- Using BFS because the graph is “just shortest path”.
- Marking a vertex visited when it is pushed into the heap.
- Forgetting to skip stale heap entries.
- Using 32-bit `int` for distances.
- Applying Dijkstra to negative edges.
- In Discount Route, allowing the discount more than once.

8 Stress testing

For small graphs, compare Dijkstra to Floyd-Warshall. For grids, compare to Floyd-Warshall on the explicit small cell graph or to a slow repeated relaxation. For Discount Route, build the two-layer graph explicitly and run a generic Dijkstra to compare against the optimized state implementation.

For Even-Hop Route, build the $2n$ parity-state graph explicitly on tiny inputs and run Floyd-Warshall over those states. This independent oracle catches the two common modeling errors: starting in odd parity and accepting the odd-parity destination state. A separate long path with one parity-changing shortcut checks the 200000-edge scale.

9 What to carry forward

Dijkstra is the default shortest-path algorithm for nonnegative weighted graphs. The modeling skill is deciding what a state must remember. Once the state graph has nonnegative edge weights, the same invariant and heap template apply.